Patient Voice Companion

An AI interviewer that helps people be understood by their clinician

System Architecture & Workflow Design

K-State AI Hackathon — Heartland AI Symposium 2026

One-line pitch. People waiting for mental-health care often cannot get an appointment soon, and when they do, they struggle to express themselves in a short, stressful visit. The Companion interviews the person at a time that suits them, in a non-judgmental conversation, and turns it into a structured, evidence-linked, *patient-approved* report that a clinician can read in about one minute.

Stack. React (frontend) · Node.js + TypeScript (backend) · PostgreSQL · open-weight LLMs hosted on the National Research Platform (NRP/Nautilus) via an OpenAI-compatible API.

Working title. Design document — version 0.1 — November 11, 2026

Contents

1 Overview **3**
1.1 Problem . . . 3
1.2 Solution in one picture . . . 3
1.3 Design principles . . . 3

2 Actors and Who Sees What **4**
2.1 Actors . . . 4
2.2 Visibility matrix . . . 4
2.3 Screens per side . . . 6

3 System Architecture **6**
3.1 Frontend (React) . . . 7
3.2 Backend (Node.js + TypeScript) . . . 7
3.3 Storage . . . 7

4 Which Model Does What **8**
4.1 Recommended call settings . . . 8

5 End-to-End Workflow **9**
5.1 Session lifecycle . . . 9
5.2 What happens on every patient message . . . 10

6 The Interview Engine **10**
6.1 Interview phases . . . 10
6.2 Coverage checklist . . . 11
6.3 Conversational screening, deterministic scoring . . . 11
6.4 Interviewer behavior rules (system prompt essentials) . . . 12

7 Safety and Crisis Protocol **12**
7.1 Two independent layers . . . 12
7.2 Response by level . . . 13

8 Report Generation Pipeline **13**
8.1 Verification rules . . . 13

8.2 Report schema (abridged) . . . 14
8.3 Two renderings of the same report . . . 14

9 Clinician Side in Detail **14**
9.1 Triage queue . . . 14
9.2 Report view — order is deliberate . . . 15
9.3 Ask the record (clinician assistant) . . . 15
9.4 Follow-up loop . . . 15

10 Data Model and API **15**
10.1 Main tables . . . 15
10.2 REST endpoints . . . 16

11 Privacy, Security and Responsible AI **16**
11.1 What we deliberately do not do . . . 17

12 Hackathon Build Plan **17**
12.1 MVP (must work in the demo) . . . 17
12.2 Stretch (in this order) . . . 17
12.3 Suggested team split . . . 17
12.4 Demo script (3 minutes) . . . 17

A Prompt Sketches **18**
A.1 Interviewer (qwen3-small) . . . 18
A.2 Safety classifier (gemma) . . . 18
A.3 Report writer (qwen3) . . . 18

1 Overview

1.1 Problem

- **Access.** There are too few psychiatrists and counselors; waiting lists are long, especially in rural areas and on campuses.
- **Expression.** Even with an appointment, many people cannot say what they mean in a 15-minute visit: anxiety in the moment, stigma, fear of being judged, forgetting what mattered, or not knowing the vocabulary clinicians use.
- **Clinician time.** Clinicians spend much of the first visit collecting basic history instead of helping.

1.2 Solution in one picture

The system has **two sides**:

- **Patient side** — a calm, voice-or-text companion that interviews the person, keeps them safe, and shows them the report *before* anyone else sees it.
- **Clinician side** — a dashboard where the clinician sees a risk-sorted queue of approved reports, validated screening scores (PHQ-9, GAD-7), the patient's own words, and every claim linked back to the transcript.

1.3 Design principles

1. An interpreter, not a lawyer

The Companion does not argue for the patient or make their case stronger. It *faithfully translates* what the patient said into clinical structure, and keeps their exact words next to every summary so the clinician can verify it. Trust from clinicians is the product.

2. Nothing about me without me

The patient sees exactly what the clinician will see, can correct or annotate it, and must approve it before submission. The only exception is an imminent safety risk, which the patient agrees to at consent (Section 7).

3. Screen and summarize — never diagnose

The system never tells anyone "you have depression". It reports symptoms, validated screening scores and severity bands, and leaves diagnosis to the clinician.

4. Safety is a separate system, not a prompt

Crisis detection runs on *every* message through its own model and a deterministic keyword layer, independent of the conversational model. If it fires, the interview stops.

5. Numbers come from code, words come from models

LLMs extract answers; deterministic code computes scores. Every quote is checked character-for-character against the transcript.

2 Actors and Who Sees What

2.1 Actors

Actor	Role
Patient	Adult preparing for, or waiting for, a mental-health visit. Talks to the Companion, reviews and approves the report.
Clinician	Psychiatrist, therapist, counselor or primary-care provider linked to the patient. Reviews reports, adds notes, requests follow-ups.
AI services	Models on NRP. They see only what the backend sends them for one task; they store nothing.
Admin (stretch)	Manages clinician accounts and patient–clinician links. Sees no clinical content.

2.2 Visibility matrix

The most important design decision: what each side can see.

Data item	Patient	Clinician	Notes
Live conversation with the Companion	✓	after submit	Clinician sees the full transcript only once the report is submitted.
Voice recordings (audio)	while recording	×	Audio is transcribed, then deleted. Only text is kept.
Draft report (AI-generated)	✓	×	Never shown to clinician until patient approves.
Patient edits & annotations	✓	✓	Shown to clinician as "patient added/corrected", with original AI text kept for audit.
Approved report	✓	✓	Identical content on both sides; different layout.
PHQ-9 / GAD-7 scores & severity band	✓	✓	Patient sees plain-language meaning ("moderate range — worth discussing").
Risk flags	✓	✓	Patient sees a gentle version ("you mentioned thoughts of not wanting to live; we've marked this so your clinician sees it first").
Evidence links (claim → transcript turn)	optional	✓	Core clinician trust feature.
Patient priorities ("what I most want help with")	✓	✓	Shown at the top of both views.
Clinician private notes	×	✓	Clinical working notes; not part of the patient report.
Follow-up questions from clinician	✓	✓	Delivered to the patient as the start of the next session.
Triage queue across patients	×	✓	Sorted by risk level, then by date.
Clinician-assistant Q&A ("ask about this patient")	×	✓	Answers grounded only in this patient's approved data.
Audit log (who viewed what, when)	own record	own actions	Full log: admin only.

2.3 Screens per side

Patient app

1. **Welcome & consent** — what this is, what it is not, who will see the report, crisis policy.
2. **Companion chat** — text box + microphone; replies can be read aloud; progress indicator ("about 10 minutes left").
3. **Crisis screen** — full-screen, appears automatically when needed.
4. **Review my report** — plain-language view, edit/annotate, approve.
5. **My history** — past reports, clinician follow-up questions.

Clinician app

1. **Triage queue** — all linked patients, red/amber/green, unread first.
2. **Report view** — risk flags → priorities → scores → summary → quotes.
3. **Transcript view** — full conversation, evidence highlights.
4. **Ask the record** — question box over this patient's data.
5. **Actions** — mark reviewed, private notes, send follow-up questions.

3 System Architecture

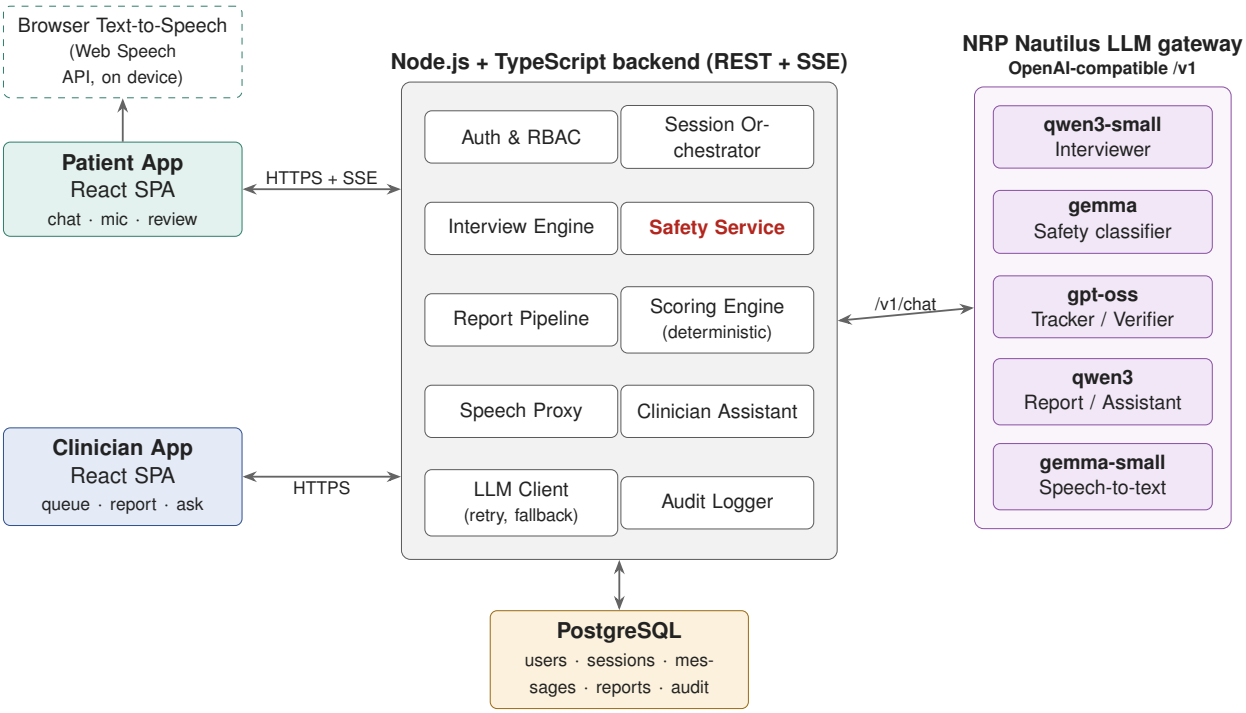


Figure 1: High-level architecture. Both frontends talk only to the backend; only the backend talks to the models.

3.1 Frontend (React)

One React app (Vite + TypeScript) with **role-based routing**: after login, patients land on `/patient/*` and clinicians on `/clinician/*`. Two separate apps are also fine, but one codebase is faster for a hackathon.

- **State & data:** React Query (TanStack) for REST calls; an `EventSource` for streamed Companion replies (Server-Sent Events).
- **UI kit:** any component library (e.g. shadcn/ui or MUI). Patient side: large type, soft colors, one action per screen. Clinician side: dense, scannable, keyboard-friendly.
- **Voice in:** `MediaRecorder` captures audio → uploaded to backend → transcribed → text is *shown to the patient to confirm or edit* before sending. Fallback: the browser's built-in speech recognition.
- **Voice out:** the browser's `speechSynthesis` reads replies aloud — no server or model needed, nothing leaves the device.

3.2 Backend (Node.js + TypeScript)

Express or Fastify, structured as modules. The frontends never call the models directly; this keeps API tokens secret and lets the backend enforce safety and permissions.

Module	Responsibility
Auth & RBAC	Login (JWT), roles `patient`/`clinician`/`admin`; every query filtered by patient–clinician link.
Session Orchestrator	Owns the session state machine (Fig. 2); decides what runs on each turn.
Interview Engine	Builds the interviewer prompt from the coverage checklist; streams the reply.
Safety Service	Keyword layer + LLM classifier on every patient message; triggers crisis protocol.
Report Pipeline	Extract → score → write → verify; produces the draft report.
Scoring Engine	Pure TypeScript functions for PHQ-9 and GAD-7 totals and bands. No AI.
Speech Proxy	Receives audio, sends to speech model, returns text, deletes audio.
Clinician Assistant	Answers clinician questions using only this patient's approved data.
LLM Client	One wrapper around the OpenAI SDK pointed at NRP: timeouts, retries, model fallback, JSON validation (zod).
Audit Logger	Append-only log of every read/write of clinical data.

3.3 Storage

PostgreSQL (or SQLite for the demo) with Prisma or Drizzle ORM. Data model in Section 10.

4 Which Model Does What

All models are served by the NRP managed LLM service at `https://ellm.nrp-nautilus.io/v1` (OpenAI-compatible; Bearer token from `nrp.ai/llmtoken`). Model names below are the NRP API names as of November 2026 — confirm against `GET /v1/models` before the build, since the catalog changes.

Role	Model	Why this model	Fallback
Interviewer (talks to patient)	`qwen3-small` 27B dense	Patient-facing replies must be fast, warm and consistent. A dense mid-size model with *thinking off* gives low latency for streaming. Status: main.	`gemma`
Safety classifier (every message)	`gemma` 31B	Different model family from the interviewer, so one model's blind spot is not the other's. Short JSON output, thinking off, temperature 0.	keyword layer alone + `qwen3-small`
Coverage tracker (after each turn)	`gpt-oss` 120B	Reads the latest exchange and updates the checklist: which topics and PHQ/GAD items are answered. Strong structured JSON. Runs in the background, so latency doesn't matter.	`qwen3-small`
Report writer (once per session)	`qwen3` 180B MoE	Most capable main model; long context fits the whole transcript; reasoning on (`medium`) for careful clinical summarization.	`gpt-oss`
Report verifier	`gpt-oss`	A *different* model checks the writer: is every claim supported by a cited transcript turn? Quotes are also checked by exact string match in code.	code-only checks
Clinician assistant	`qwen3`	Answers "has the patient mentioned sleep problems before?" over the full transcript and past reports; 1M-token context means no RAG is needed for the demo.	`gpt-oss`
Speech-to-text	`gemma-small` 12B, audio input	The only NRP model with audio input. Status: *evaluating*, so we keep a fallback.	Browser Web Speech API
Text-to-speech	none (browser)	`speechSynthesis` in the browser: free, instant, private.	—
Embeddings (stretch)	`qwen3-embedding`	Only needed later for searching many sessions per patient.	—

4.1 Recommended call settings

Role	Temp.	Thinking	Output
Interviewer	0.6	off	Streamed text, 1–3 sentences, one question at a time
Safety	0.0	off	JSON: `{level, categories, evidence}`
Tracker	0.1	low	JSON: checklist updates with turn IDs
Report writer	0.2	medium	JSON matching the report schema (Section 8)
Verifier	0.0	low	JSON: list of unsupported claims
Assistant	0.3	low	Text with turn-ID citations

NRP-specific notes

- Thinking is toggled via `extra_body.chat_template_kwargs` (e.g. `enable_thinking: false`).
- Send a random per-session `cache_salt` (256-bit, base64) so prompts are never cache-shared across users.
- Set `max_tokens` explicitly (well below the context size). Respect NRP fair-use concurrency limits; check `nrp.ai/llm-status` before the demo.
- Keep model names in one config file so they can be swapped if the endpoint list changes.

5 End-to-End Workflow

5.1 Session lifecycle

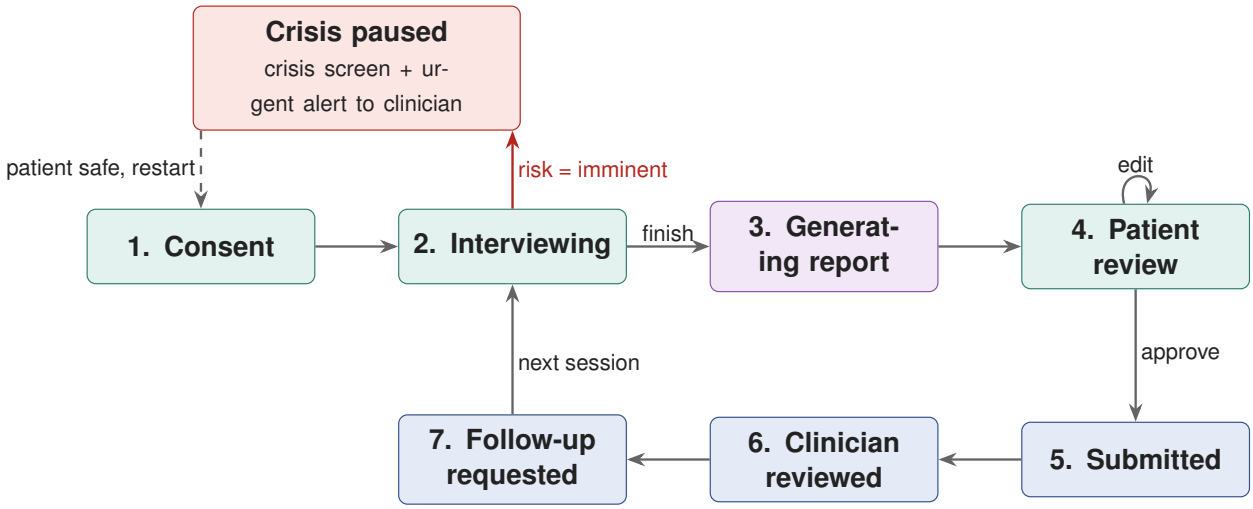


Figure 2: Session state machine owned by the Session Orchestrator. Green = patient side, blue = clinician side, purple = AI, red = safety.

1. **Consent.** Patient reads a short, plain explanation: this is an AI, it is not therapy or a diagnosis, the report goes to the named clinician only after approval, and if there is an immediate safety risk, the care team will be alerted. Checkbox to continue.
2. **Interviewing.** Conversation (Section 6). Patient can pause and resume later; the session is saved.
3. **Generating report.** Report pipeline runs (about 20–60 seconds).
4. **Patient review.** Patient sees the report in plain language, can correct facts, add context ("I said 'every day' but I meant most days"), or remove things they don't want shared (removals are noted as "patient chose not to share this topic").
5. **Submitted.** Report is locked and appears in the clinician's queue.

6. **Clinician reviewed.** Clinician reads, adds private notes, marks as reviewed.
7. **Follow-up requested.** Clinician can send 1–3 questions; the Companion opens the patient's next session with them.

5.2 What happens on every patient message

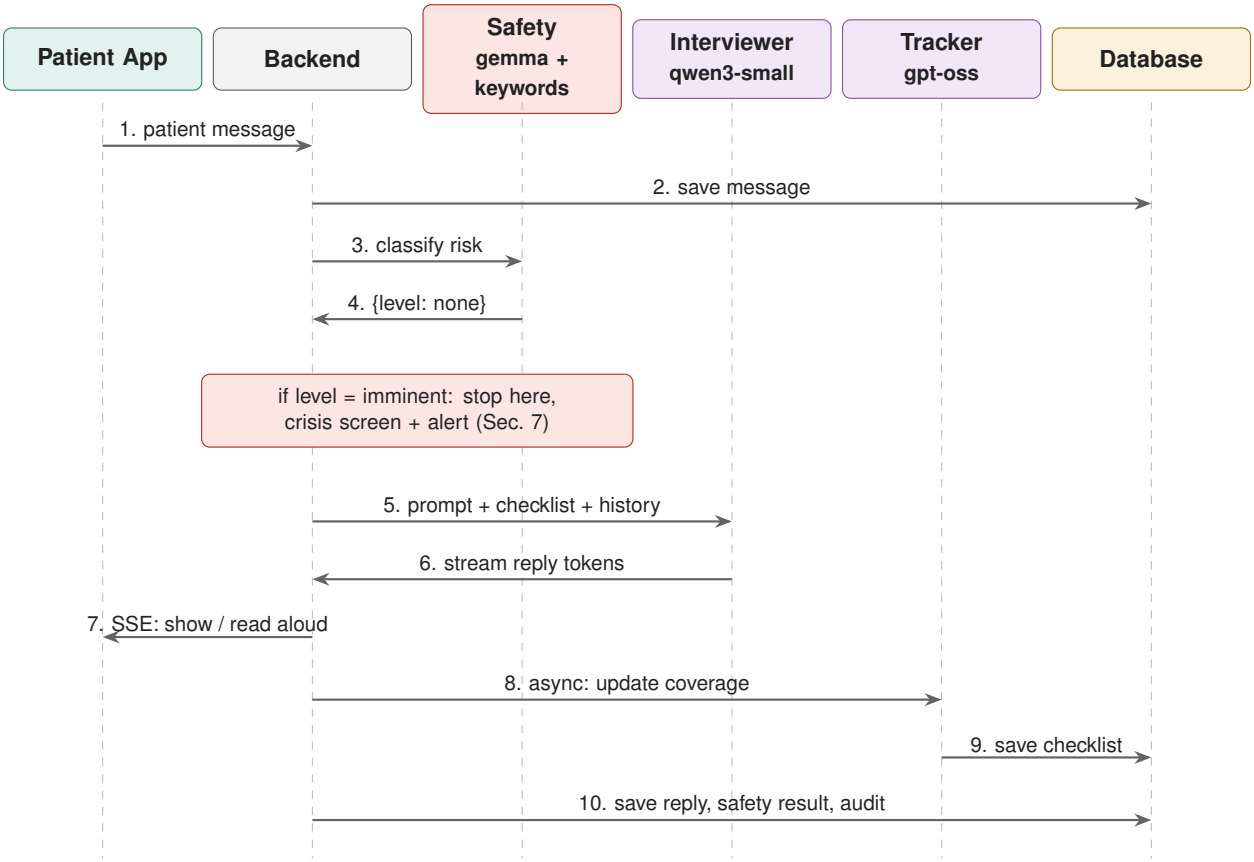


Figure 3: Per-turn sequence. Safety runs *before* the interviewer replies; the tracker runs *after*, in the background.

Latency budget: safety (~0.5–1 s) + first interviewer token (~1 s). A deterministic keyword match (e.g. "kill myself", "end it", "overdose") triggers the crisis path *immediately*, without waiting for any model.

6 The Interview Engine

6.1 Interview phases

The interviewer does not ask a rigid questionnaire. It follows a semi-structured clinical interview, guided by a **coverage checklist** that the tracker keeps up to date.

1. **Welcome** — name preference, how they're feeling right now, reassurance (no right or wrong answers, can skip anything).

2. **Open story** — "What's been going on that made you want to talk to someone?" Let the patient talk; reflect back.
3. **Guided exploration** — the checklist domains below, in whatever order follows naturally from the story.
4. **Screening items** — PHQ-9 and GAD-7 woven into the conversation, then *confirmed* on the standard scale.
5. **Priorities** — "If your clinician could only help with one thing, what would it be?" and "Is there anything you find hard to say in person?"
6. **Wrap-up** — summary read back to the patient ("Here's what I heard. . . did I get that right?"), then finish.

6.2 Coverage checklist

Domain	What is captured
Chief concern	In the patient's words; when it started; what triggered it
Mood & anxiety	Low mood, worry, panic, irritability; frequency and duration
Sleep, appetite, energy	Changes, how long
Concentration & functioning	Work/school, home, relationships, daily tasks
Substance use	Alcohol, cannabis, other; changes recently
Safety	Thoughts of self-harm or suicide (PHQ-9 item 9), harm to others; handled per Section 7
History	Past diagnoses, treatment, therapy, hospitalizations, medications
Support & strengths	People they rely on, what helps, coping that works
Goals & preferences	What they want from care (talk therapy, medication, both, unsure)

6.3 Conversational screening, deterministic scoring

PHQ-9 and GAD-7 are public-domain, widely used, and instantly recognized by clinicians. Each item uses the scale *over the last 2 weeks: not at all (0), several days (1), more than half the days (2), nearly every day (3)*.

- The interviewer asks naturally ("How has your sleep been lately?").
- If the tracker can't confidently map the answer to 0–3, the interviewer asks a short confirmation ("Would you say that's several days, more than half the days, or nearly every day?"). The patient can also tap the answer.
- The tracker stores each item as `{item, value, confidence, turn_id}`.
- The **Scoring Engine** (plain code) computes totals and bands. Missing items are reported as missing, never guessed.

Scale	Range	Severity bands
PHQ-9	0–27	0–4 minimal, 5–9 mild, 10–14 moderate, 15–19 moderately severe, 20–27 severe
GAD-7	0–21	0–4 minimal, 5–9 mild, 10–14 moderate, 15–21 severe

6.4 Interviewer behavior rules (system prompt essentials)

- One question at a time; 1–3 short sentences; warm, plain language; no jargon.
- Reflect before asking ("That sounds exhausting."). Never judge, never argue, never give medical advice or diagnoses.
- The patient can skip any question; skipped topics are recorded as "declined", not as negative answers.
- Never promise confidentiality beyond the consent terms.
- If asked "what's wrong with me?" — explain that the clinician is the right person to answer, and that the report will help them.

7 Safety and Crisis Protocol

Safety is non-negotiable

The Companion is not a crisis service. Its job in a crisis is to stop, show real help immediately, and alert a human.

7.1 Two independent layers

1. **Keyword layer (code).** A curated phrase list. Instant. High recall; may over-trigger, which is acceptable.
2. **Classifier (`gemma`).** Returns:

```
{ "level": "none" | "concern" | "elevated" | "imminent",
  "categories": ["suicidal_ideation", "self_harm", "harm_to_others",
                 "abuse", "psychosis", "substance_emergency"],
  "evidence": "exact phrase from message" }
```

The final level is the **higher** of the two layers.

7.2 Response by level

Level	Patient side	Clinician side
none	Conversation continues.	—
concern	Interviewer gently acknowledges; resources link shown in footer.	Noted in report.
elevated e.g. passive thoughts of death, hopelessness	Interviewer asks safety follow-ups calmly; crisis resources shown in a persistent banner.	Red flag at the top of the report; patient marked high priority in queue.
imminent plan, intent, means, or timeline	**Interview stops.** Full-screen crisis page: call or text **988**, text HOME to **741741**, call **911**. Calm message, no further questions.	**Urgent alert** sent immediately (as agreed at consent), even before report approval: risk level, evidence phrase, time.

For the hackathon, the "urgent alert" is a red banner on the clinician dashboard plus (stretch) an email or SMS.

8 Report Generation Pipeline

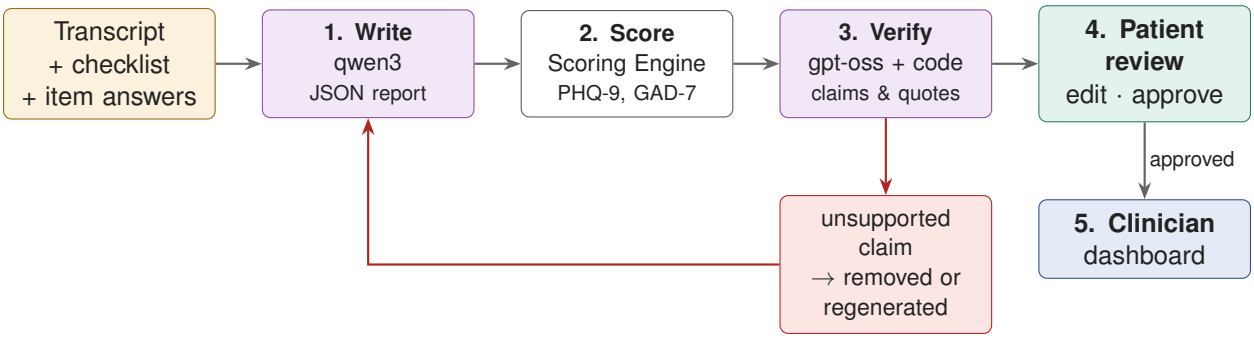


Figure 4: Report pipeline. Models write words; code computes numbers and checks quotes; a second model checks the first.

8.1 Verification rules

- Every summary statement must cite at least one `turn_id`. The verifier (`gpt-oss`) checks each claim against its cited turns.
- Every quote must appear *verbatim* in the transcript (exact string match in code). Failing quotes are dropped.
- Scores in the report are overwritten by the Scoring Engine's output, never taken from the LLM.
- Maximum one regeneration; after that, unsupported claims are removed and a note is added.

8.2 Report schema (abridged)

```
{
  "patient": { "preferred_name": "Sam", "age_range": "25-34" },
  "session": { "id": "s_123", "date": "2026-11-11", "duration_min": 18 },
  "risk": { "level": "elevated",
            "flags": [{ "category": "suicidal_ideation", "detail": "passive thoughts, no plan",
                        "turn_ids": [14, 15] }] },
  "patient_priorities": ["Sleeping again", "Not feeling alone at work"],
  "screening": {
    "phq9": { "items": [2,3,2,1,1,2,2,0,1], "total": 14, "band": "moderate", "missing": [] },
    "gad7": { "items": [2,2,1,2,1,1,1], "total": 10, "band": "moderate", "missing": [] }
  },
  "summary": {
    "chief_concern": { "text": "Low mood and poor sleep since job change in August.", "turn_ids":
        [3,4] },
    "history_present": [ { "text": "...", "turn_ids": [5,6,9] } ],
    "functioning": [ { "text": "...", "turn_ids": [11] } ],
    "substance_use": [ { "text": "...", "turn_ids": [12] } ],
    "past_history": [ { "text": "...", "turn_ids": [13] } ],
    "supports": [ { "text": "...", "turn_ids": [16] } ],
    "care_preferences":[ { "text": "...", "turn_ids": [18] } ]
  },
  "quotes": [ { "text": "I lie awake replaying everything I did wrong.", "turn_id": 6 } ],
  "declined_topics": ["substance_use"],
  "patient_annotations": [ { "section": "functioning", "note": "..." } ],
  "provenance": { "models": {"writer":"qwen3","verifier":"gpt-oss"}, "ai_generated": true }
}
```

8.3 Two renderings of the same report

Patient view	Clinician view
Plain language, second person (“You told me…”)	Clinical language, third person, SOAP-like headings
Scores explained in words (“moderate range — this is something worth talking about”)	Scores with item breakdown and band
Edit, annotate, remove buttons on every section	Evidence chips: click to jump to transcript turn
Big “Send to my clinician” button	Risk flags pinned at top; “mark reviewed”, notes, follow-up

9 Clinician Side in Detail

9.1 Triage queue

Sorted by (1) risk level, (2) unreviewed first, (3) newest. Each row: patient name, date, risk badge, PHQ-9/GAD-7 totals with trend arrow (if previous sessions exist), top patient priority.

9.2 Report view — order is deliberate

1. **Risk flags** with evidence phrase and turn link.
2. **Patient priorities** — in their words.
3. **Screening scores** — totals, bands, item breakdown, missing items.
4. **Clinical summary** — each line with evidence chips.
5. **Patient's own words** — 3–6 verified quotes.
6. **Patient annotations & declined topics.**
7. **Full transcript** (collapsible).

A permanent banner reads: *"AI-generated summary of a patient-approved conversation. Not a diagnosis. Verify against transcript."*

9.3 Ask the record (clinician assistant)

The clinician types a question; the backend sends `qwen3` only this patient's approved reports and transcripts, with the instruction to answer only from them and cite turn IDs, or say "not discussed". Examples: "Has the patient mentioned medication side effects?" · "How has sleep changed across sessions?"

9.4 Follow-up loop

Clinician writes up to three questions → stored as `FollowUpRequest` → patient sees a notification → the next Companion session opens with those questions, then continues normally. This turns a one-off intake into ongoing, between-visit care.

10 Data Model and API

10.1 Main tables

Table	Key fields
`users`	id, role, name, email, password_hash
`care_links`	patient_id, clinician_id, status
`sessions`	id, patient_id, state, started_at, ended_at, cache_salt
`messages`	id, session_id, turn_id, sender (patient/companion), text, input_mode (text/voice), created_at
`safety_events`	id, session_id, turn_id, level, categories, evidence, source (keyword/model), alerted_at
`checklist_items`	session_id, item_key (e.g. `phq9_3`, `sleep`), value, confidence, turn_ids
`reports`	id, session_id, version (ai_draft / patient_approved), json, approved_at
`report_edits`	report_id, section, original_text, new_text, kind (correct/annotate/remove)
`clinician_notes`	report_id, clinician_id, text (private)
`follow_ups`	patient_id, clinician_id, questions[], status
`audit_log`	actor_id, action, resource, timestamp

10.2 REST endpoints

Method	Path	Purpose
Patient		
POST	`/api/sessions`	Start session (after consent)
POST	`/api/sessions/:id/messages`	Send message; response is an SSE stream (reply tokens + safety events)
POST	`/api/speech/transcribe`	Upload audio, receive text (audio deleted)
POST	`/api/sessions/:id/finish`	Run report pipeline
GET	`/api/reports/:id/patient-view`	Draft report in patient language
PATCH	`/api/reports/:id`	Edits and annotations
POST	`/api/reports/:id/approve`	Lock and submit to clinician
GET	`/api/me/follow-ups`	Pending clinician questions
Clinician		
GET	`/api/clinician/queue`	Risk-sorted list of approved reports
GET	`/api/clinician/reports/:id`	Clinician view + transcript
POST	`/api/clinician/reports/:id/review`	Mark reviewed, add private note
POST	`/api/clinician/patients/:id/ask`	Clinician assistant question
POST	`/api/clinician/patients/:id/follow-ups`	Send follow-up questions
GET	`/api/clinician/alerts`	Live urgent safety alerts (SSE)

11 Privacy, Security and Responsible AI

- **Open-weight models on research infrastructure.** Conversations go to NRP-hosted models, not to commercial AI providers. Per-session `cache_salt` prevents cross-user prompt caching.
- **Minimum data.** Audio deleted after transcription. No real patient data during the hackathon — demo personas only.
- **Access control.** Clinicians can only see patients linked to them, and only approved reports (except urgent safety alerts).
- **Transparency.** Every AI output is labeled; model names are stored in the report's provenance.
- **No diagnosis, no treatment advice.** Enforced in prompts and checked by the verifier.
- **Bias & fairness.** Test personas across ages, genders (including trans and nonbinary people), cultures, and ways of speaking (short answers, non-native English, dialect) to check the summary stays faithful.
- **Not production-ready for real patients.** A real deployment would need HIPAA compliance, clinical validation, IRB review and a formal crisis-escalation agreement. We state this openly.

11.1 What we deliberately do not do

- We do not diagnose or recommend medication.
- We do not replace a clinician or act as a crisis line.
- We do not send anything to a clinician without the patient's approval (except imminent risk, agreed at consent).
- We do not let the AI compute scores or invent quotes.
- We do not keep voice recordings.

12 Hackathon Build Plan

12.1 MVP (must work in the demo)

1. Patient: consent → text chat with streaming replies → finish.
2. Safety keyword layer + `gemma` classifier → crisis screen.
3. Report pipeline: `qwen3` writer + deterministic PHQ-9/GAD-7 scoring + quote check.
4. Patient review & approve.
5. Clinician: queue + report view with risk flags and scores.

12.2 Stretch (in this order)

Voice input (`gemma-small`) and read-aloud → evidence chips linked to transcript → verifier model → clinician assistant → follow-up loop → score trends across sessions.

12.3 Suggested team split

Person	Owns
A (backend)	Node API, LLM client, interviewer + safety + tracker calls, SSE streaming
B (frontend)	React patient chat, review page, clinician dashboard
C (domain / product)	Prompts and checklist wording, demo personas, testing, safety phrase list, slides, demo script

12.4 Demo script (3 minutes)

1. 20 seconds: a real-feeling story — someone on a 3-month waitlist who freezes up in appointments.
2. 60 seconds: the persona talks to the Companion (voice if ready); show warmth and one confirmation question.

3. 30 seconds: patient reviews the report, corrects one detail, approves.
4. 40 seconds: switch to clinician view — risk flag, scores, quote, click an evidence chip to the transcript.
5. 30 seconds: “What we deliberately don’t do” slide, and the NRP-hosted privacy story.

A Prompt Sketches

A.1 Interviewer (qwen3-small)

```
You are a warm, calm companion helping {preferred_name} prepare for a visit with
their mental-health clinician. You are not a therapist and you do not diagnose.
Ask ONE short question at a time (1-3 sentences). Reflect what you heard first.
Use plain words. The person may skip anything. Never give medical advice.
Topics still to cover (in priority order): {uncovered_checklist}
Screening items needing a frequency answer: {pending_items}
If an answer is vague, gently offer: "not at all, several days, more than half
the days, or nearly every day?"
When everything is covered, read back a short summary and ask if it is right.
```

A.2 Safety classifier (gemma)

```
Classify the risk in the PATIENT MESSAGE below. Consider the recent context.
Return ONLY JSON: {"level": "none|concern|elevated|imminent",
"categories": [...], "evidence": "<exact phrase or empty>"}.
imminent = current plan, intent, means, or timeline to harm self/others.
elevated = thoughts of death or self-harm without plan; severe hopelessness.
When uncertain between two levels, choose the higher one.
```

A.3 Report writer (qwen3)

```
Write a clinician-facing intake summary from the transcript and checklist.
Rules: every statement must cite turn_ids; use only what the patient said;
no diagnoses; mark unknowns as "not discussed"; quotes must be copied exactly.
Do NOT compute screening totals. Output JSON matching the schema.
```